1) `$exists`

Meaning

Check karta hai field **document me present hai ya nahi.**

A) Single field

जिन students me `fees` field present ho

JavaScript

```
db.students.find({ fees: { $exists: true } })
```

जिन employees me `bonus` field present na ho

JavaScript

```
db.employees.find({ bonus: { $exists: fals
```

B) Array field

Jis post me `tags` field ho

JavaScript

```
db.posts.find({ tags: { $exists: true } })
```

C) Embedded field

Jis post me `comments.user` field ho

JavaScript

```
db.posts.find({ "comments.user": { $exists: true } })
```

2) `$type`

Meaning

Field ka **BSON type** check karta hai.

Common types:

- `"string"`
- `"int"`
- `"double"`
- `"bool"`
- `"date"`
- `"array"`
- `"object"`

A) Single field

students me name string type ho

JavaScript

```
db.students.find({ name: { $type: "string" } })
```

employees me salary number/double type ho

JavaScript

```
db.employees.find({ salary: { $type: "double" } })
```

orders me orderDate date type ho

JavaScript

```
db.orders.find({ orderDate: { $type: "date"
```

B) Array field

posts me tags array type ho

JavaScript

```
db.posts.find({ tags: { $type: "array" } })
```

C) Embedded field

comments.user string type ho

JavaScript

```
db.posts.find({ "comments.user": { $type: "string" } })
```

3) `$regex`

Meaning

Pattern matching / text search ke liye.

A) Single field

Students jinka name Rahul se start hota ho

JavaScript

```
db.students.find({ name: { $regex: "^Rahul" } })
```

Employees jinki company me “Tech” ho

JavaScript

```
db.employees.find({ company: { $regex: "Tech" } })
```

Users jinka email gmail.com par end hota ho

JavaScript

```
db.users.find({ email: { $regex: "gmail\\.com$" } })
```

B) Array field

tags me nodejs word ho

JavaScript

```
db.posts.find({ tags: { $regex: "node" } })
```

> Array string values par regex tab kaam karta hai jab eleme

C) Embedded field

comment user Rahul ho / Rahul se start hota ho

JavaScript

```
db.posts.find({ "comments.user": { $regex: "^Rahul" } })
```

4) `$size`

Meaning

Array ka exact size check karta hai.

A) Single field

❌ single normal field par use nahi hota.

B) Array field

Jis post me exactly 3 tags ho

JavaScript

```
db.posts.find({ tags: { $size: 3 } })
```

C) Embedded field

❌ single embedded field par direct use nahi hota.

D) Embedded array

Jis post me exactly 2 comments ho

JavaScript

```
db.posts.find({ comments: { $size: 2 } })
```

5) `$all`

Meaning

Array me **saari given values present honi chahiye.**

A) Single field

❌ single field par use nahi hota.

B) Array field

Jis post me mongodb aur nodejs dono tags ho

JavaScript

```
db.posts.find({
  tags: { $all: ["mongodb", "nodejs"] }
})
```

C) Embedded field

Direct single embedded field par नहीं.

D) Embedded array

Agar embedded array ke kisi field ke values check karne ho to usually dot notation ya `$elemMatch` better hota hai.

---

6) `$elemMatch`

Meaning

Array ke **same element/document** par multiple conditions lagani ho to use hota hai.

Ye bahut important hai interview me.

A) Single field

❌ single field par use nahi hota.

B) Array field

Agar numeric array hota to useful hota.

Tumhare `tags` string array me simple use ka example:

Agar array me exact mongodb ho

JavaScript

```
db.posts.find({
  tags: { $elemMatch: { $eq: "mongodb" } }
})
```

> Practical use tags ke liye `$in` / direct equality jyada common hai.

C) Embedded field

Direct single embedded field par नहीं.

D) Embedded array (most important)

`comments` array of objects hai:

JavaScript

```
comments: [
  { user: "Rahul", msg: "Nice" },
  { user: "Priya", msg: "Good" }
]
```

Jis post me same comment object me user Rahul aur msg Nice ho

JavaScript

```
db.posts.find({
  comments: {
    $elemMatch: {
      user: "Rahul",
      msg: "Nice"
    }
  }
})
```

Jis post me same comment object me user Priya aur msg Good ho

JavaScript

```
db.posts.find({
  comments: {
    $elemMatch: {
      user: "Priya",
      msg: "Good"
    }
```

Important difference: dot notation vs `$elemMatch`

Dot notation

JavaScript

```
db.posts.find({
  "comments.user": "Rahul",
  "comments.msg": "Good"
})
```

Ye **different elements** se bhi match ho sakta hai:

- ek comment me user Rahul
- doosre comment me msg Good

`$elemMatch`

JavaScript

```
db.posts.find({
  comments: {
    $elemMatch: {
      user: "Rahul",
      msg: "Good"
    }
  }
})
```

Ye ensure karta hai ki **same array object** me Rahul aur Good dono ho.

```
// $exists
db.students.find({ fees: { $exists: true } })
db.posts.find({ "comments.user": { $exists: true } })

// $type
db.students.find({ name: { $type: "string" } })
db.posts.find({ tags: { $type: "array" } })
db.posts.find({ "comments.msg": { $type: "string" } })

// $regex
db.students.find({ name: { $regex: "^Rahul" } })
db.users.find({ email: { $regex: "gmail\\.com$" } })
db.posts.find({ "comments.user": { $regex: "^Rahul" } })

// $size
db.posts.find({ tags: { $size: 3 } })
db.posts.find({ comments: { $size: 2 } })
```

```
// $all
db.posts.find({ tags: { $all: ["mongodb", "nodejs"] } })

// $elemMatch
db.posts.find({
  comments: {
    $elemMatch: { user: "Rahul", msg: "Nice" }
  }
})
```

Sabse important yaad rakhne wali baatein

- `$exists` → field hai ya nahi
- `$type` → field ka BSON type
- `$regex` → pattern matching
- `$size` → exact array length
- `$all` → array me saari values honi chahiye
- `$elemMatch` → same embedded array object par multiple conditions

Iska use tab hota hai jab **document ke andar 2 fields ko aapas me compare** karna ho, ya **aggregation expressions ko `find()` query ke andar use** karna ho.

Simple words me:

- normal query me hum likhte hain:

```javascript
{ salary: { $gt: 50000 } }
```

- but agar compare karna ho:
  salary > bonus
  to normal query se nahi hoga, waha `$expr` use karte hain.

A) Employees jinki salary bonus se zyada ho

```javascript
db.employees.find({
  $expr: { $gt: ["$salary", "$bonus"] }
})
```

B) Products jahan stock = rating

```javascript
db.products.find({
  $expr: { $eq: ["$stock", "$rating"] }
})
```

2) Arithmetic expression inside `$expr`

A) Employees jinki salary bonus se कम से कम 50000 ज्यादा ho

JavaScript

```
db.employees.find({
  $expr: {
    $gt: [
      { $subtract: ["$salary", "$bonus"] },
      50000
    ]
  }
})
```

B) Orders jahan totalAmount = quantity * price

JavaScript

```
db.orders.find({
  $expr: {
    $eq: [
      "$totalAmount",
      { $multiply: ["$quantity", "$price"] }
    ]
  }
})
```

C) Payments jahan salary + bonus > 100000

JavaScript

```
db.payments.find({
  $expr: {
    $gt: [
      { $add: ["$salary", "$bonus"] },
      100000
    ]
```

3) Single field + constant with `$expr`

Students jinki fees 50000 se zyada ho

JavaScript

```
db.students.find({
  $expr: { $gt: ["$fees", 50000] }
})
```

> Ye normal query se bhi ho jayega:

JavaScript

```
db.students.find({ fees: { $gt: 50000 } })
```

But `$expr` ka main use **field-vs-field** aur **expression** me hota hai.

-929e-0780ef2b4e4f

4) Array field with `$expr`

Tumhare `posts.tags` array hai.

Jis post me tags ka size 3 ho

JavaScript

```
db.posts.find({
  $expr: { $eq: [ { $size: "$tags" }, 3 ] }
})
```

Jis post me tags count 2 se zyada ho

JavaScript

```
db.posts.find({
  $expr: { $gt: [ { $size: "$tags" }, 2 ] }
})
```

5) Embedded array with `$expr`

Tumhare `posts.comments` embedded array hai.

Jis post me comments ka size 2 ho

JavaScript

```
db.posts.find({
  $expr: { $eq: [ { $size: "$comments" }, 2 ] }
})
```

Jis post me comments 1 se zyada ho

JavaScript

```
db.posts.find({
  $expr: { $gt: [ { $size: "$comments" }, 1 ] }
})
```

6) Date field with `$expr`

Orders jinka month > 6 ho

JavaScript

```
db.orders.find({
  $expr: {
    $gt: [
      { $month: "$orderDate" },
      6
    ]
  }
})
```

7) Multiple conditions inside `$expr`

Employees jahan salary > bonus AND age > 30

JavaScript

```
db.employees.find({
  $expr: {
    $and: [
      { $gt: ["$salary", "$bonus"] },
      { $gt: ["$age", 30] }
    ]
  }
})
```

Orders jahan totalAmount = quantity * price AND status = Delivered

JavaScript

```
db.orders.find({
  $expr: {
    $and: [
      {
        $eq: [
          "$totalAmount",
          { $multiply: ["$quantity", "$price"] }
        ]
      },
      { $eq: ["$status", "Delivered"] }
    ]
  }
})
```

9) `$expr` me common operators

`$expr` ke andar ye sab aa sakte hain:

- comparison:
  - `$eq`
  - `$ne`
  - `$gt`
  - `$gte`
  - `$lt`
  - `$lte`
- arithmetic:
  - `$add`
  - `$subtract`
  - `$multiply`
  - `$divide`
  - `$mod`

- arithmetic:
  - `$add`
  - `$subtract`
  - `$multiply`
  - `$divide`
  - `$mod`
- logical:
  - `$and`
  - `$or`
  - `$not`
- date:
  - `$year`
  - `$month`
  - `$dayOfMonth`
- array:
  - `$size`

```
// field vs field
db.employees.find({
  $expr: { $gt: ["$salary", "$bonus"] }
})

db.orders.find({
  $expr: {
    $eq: [
      "$totalAmount",
      { $multiply: ["$quantity", "$price"] }
    ]
  }
})

// field + arithmetic
db.payments.find({
  $expr: {
    $gt: [
      { $add: ["$salary", "$bonus"] },
      100000
    ]
  }
```

```
// array size
db.posts.find({
  $expr: { $eq: [ { $size: "$tags" }, 3 ] }
})

db.posts.find({
  $expr: { $gt: [ { $size: "$comments" }, 1 ] }
})

// date expression
db.orders.find({
  $expr: {
    $gt: [ { $month: "$orderDate" }, 6 ]
  }
})
```

MongoDB Aggregation Pipeline

Use:

JavaScript

```
use companyDB
```

Syntax

JavaScript

```
db.collection.aggregate([
  { stage1 },
  { stage2 },
  { stage3 }
])
```

Aggregation me documents **pipeline ke through pass** hote hain.

1) `$match`

Meaning

Filtering stage.

`find()` jaisa kaam karta hai.

Example 1: Indore students

JavaScript

```
db.students.aggregate([
  { $match: { city: "Indore" } }
])
```

Example 2: salary > 100000 employees

Example 2: salary > 100000 employees

JavaScript

```
db.employees.aggregate([
  { $match: { salary: { $gt: 100000 } } }
])
```

Example 3: Delivered orders

JavaScript

```
db.orders.aggregate([
  { $match: { status: "Delivered" } }
])
```

2) `$project`

Meaning

Output me **kaunse fields chahiye** ye decide karta hai.
New computed fields bhi bana sakte ho.

Example 1: students me sirf name, city, fees

JavaScript

```
db.students.aggregate([
  {
    $project: {
      _id: 0,
      name: 1,
      city: 1,
      fees: 1
    }
  }
])
```

Example 2: employees me total income = salary + bonus

JavaScript

```
db.employees.aggregate([
  {
    $project: {
      _id: 0,
      name: 1,
      salary: 1,
      bonus: 1,
      totalIncome: { $add: ["$salary", "$bonus"] }
    }
  }
])
```

Example 3: orders me totalAmount aur month

JavaScript

```
db.orders.aggregate([
  {
    $project: {
      _id: 0,
      customerName: 1,
      totalAmount: 1,
      orderMonth: { $month: "$orderDate" }
    }
  }
])
```

3) `$group`

Meaning

Grouping + aggregation ke liye.
SQL ke `GROUP BY` jaisa.

Common accumulators:

- `$sum`
- `$avg`
- `$max`
- `$min`
- `$push`
- `$addToSet`

Example 1: city-wise student count

JavaScript

```
db.students.aggregate([
  {
    $group: {
      _id: "$city",
      totalStudents: { $sum: 1 }
    }
  }
])
```

Example 2: department-wise avg salary

JavaScript

```
db.employees.aggregate([
  {
    $group: {
      _id: "$department",
      avgSalary: { $avg: "$salary" }
    }
  }
```

Example 3: company-wise max salary

JavaScript

```
db.employees.aggregate([
  {
    $group: {
      _id: "$company",
      maxSalary: { $max: "$salary" }
    }
  }
])
```

Example 4: city-wise total orders amount

JavaScript

```
db.orders.aggregate([
  {
    $group: {
      _id: "$city",
      totalRevenue: { $sum: "$totalAmount"
    }
```

4) `$sort`

Meaning

Documents ko sort karta hai.

- `1` → ascending
- `-1` → descending

Example 1: students by fees ascending

JavaScript

```
db.students.aggregate([
  { $sort: { fees: 1 } }
])
```

Example 2: employees by salary descending

JavaScript

```
db.employees.aggregate([
  { $sort: { salary: -1 } }
])
```

Example 3: posts by likes descending

JavaScript

```
db.posts.aggregate([
  { $sort: { likes: -1 } }
])
```

5) `$limit`

Meaning

Sirf first N documents return karega.

Example: top 5 highest salary employees

JavaScript

```
db.employees.aggregate([
  { $sort: { salary: -1 } },
  { $limit: 5 }
])
```

---

6) `$skip`

Meaning

First N documents skip karta hai.

Example: first 5 skip karke next 5 students

JavaScript

```
db.students.aggregate([
  { $skip: 5 },
  { $limit: 5 }
])
```

7) `$unwind`

Meaning

Array ke har element ko **alag document** bana deta hai.

Tumhare `posts.tags` aur `posts.comments` par bahut useful hai.

Example 1: tags unwind

JavaScript

```
db.posts.aggregate([
  { $unwind: "$tags" }
])
```

Example 2: comments unwind

JavaScript

```
db.posts.aggregate([
  { $unwind: "$comments" }
])
```

Example 3: हर tag ka count

JavaScript

```
db.posts.aggregate([
  { $unwind: "$tags" },
  {
    $group: {
      _id: "$tags",
      count: { $sum: 1 }
    }
```

Example 4: हर comment user ka count

JavaScript

```
db.posts.aggregate([
  { $unwind: "$comments" },
  {
    $group: {
      _id: "$comments.user",
      totalComments: { $sum: 1 }
    }
  }
])
```

8) `$lookup`

Meaning

Ek collection ko doosri collection se join karta hai.
SQL JOIN jaisa.

Tumhare data me good example:

- `orders.productName`
- `products.name`

Example: orders + products join

JavaScript

```
db.orders.aggregate([
  {
    $lookup: {
      from: "products",
      localField: "productName",
      foreignField: "name",
      as: "productDetails"
    }
  }
])
```

`$lookup` fields meaning

- `from` → kis collection se join karna hai
- `localField` → current collection ka field
- `foreignField` → dusri collection ka m ng field
- `as` → joined result kis field me aayega

)29e-0780ef2b4e4f

Example 2: orders + products + project

JavaScript

```
db.orders.aggregate([
  {
    $lookup: {
      from: "products",
      localField: "productName",
      foreignField: "name",
      as: "productDetails"
    }
  },
  {
    $project: {
      _id: 0,
      customerName: 1,
      productName: 1,
      totalAmount: 1,
      productDetails: 1
    }
  }
])
```

Combined pipeline examples

1) Indore students sorted by fees descending

JavaScript

```
db.students.aggregate([
  { $match: { city: "Indore" } },
  { $sort: { fees: -1 } }
])
```

2) Department-wise average salary, highest first

JavaScript

```
db.employees.aggregate([
  {
    $group: {
      _id: "$department",
      avgSalary: { $avg: "$salary" }
    }
  },
  { $sort: { avgSalary: -1 } }
])
```

3) Top 3 cities by total order revenue

JavaScript

```javascript
db.orders.aggregate([
  {
    $group: {
      _id: "$city",
      revenue: { $sum: "$totalAmount" }
    }
  },
  { $sort: { revenue: -1 } },
  { $limit: 3 }
])
```

4) Tag-wise post count

JavaScript

```
db.posts.aggregate([
  { $unwind: "$tags" },
  {
    $group: {
      _id: "$tags",
      totalPosts: { $sum: 1 }
    }
  },
  { $sort: { totalPosts: -1 } }
])
```

5) Orders whose totalAmount > 5000, then only customer + amount

JavaScript

```
db.orders.aggregate([
  { $match: { totalAmount: { $gt: 5000 } } },
  {
    $project: {
      _id: 0,
      customerName: 1,
      totalAmount: 1
    }
  }
])
```

```
// $match
db.students.aggregate([
  { $match: { city: "Indore" } }
])

// $project
db.employees.aggregate([
  {
    $project: {
      _id: 0,
      name: 1,
      totalIncome: { $add: ["$salary", "$bonus"] }
    }
  }
])
```

```
// $group
db.employees.aggregate([
  {
    $group: {
      _id: "$department",
      avgSalary: { $avg: "$salary" }
    }
  }
])

// $sort + $limit
db.employees.aggregate([
  { $sort: { salary: -1 } },
  { $limit: 5 }
])

// $skip
db.students.aggregate([
  { $skip: 5 },
  { $limit: 5 }
])
```

```
// $unwind
db.posts.aggregate([
  { $unwind: "$tags" },
  { $group: { _id: "$tags", count: { $sum: 1 } } }
])

// $lookup
db.orders.aggregate([
  {
    $lookup: {
      from: "products",
      localField: "productName",
      foreignField: "name",
      as: "productDetails"
    }
  }
])
```

Interview one-line definitions

- `$match` → filter documents
- `$project` → choose/compute fields
- `$group` → group + aggregate
- `$sort` → sort documents
- `$limit` → first N docs
- `$skip` → skip first N docs
- `$unwind` → array elements ko separate documents me todna
- `$lookup` → another collection se join

1) `$addFields`

Meaning

Existing document me **new field add** karta hai, ya existing field ko modify karke naya output deta hai.

> `$project` me fields select bhi hote hain
> `$addFields` me mostly **extra field add** hota hai.

A) Employees me totalIncome = salary + bonus

JavaScript

```
db.employees.aggregate([
  {
    $addFields: {
      totalIncome: { $add: ["$salary", "$bonus"] }
    }
  }
```

B) Orders me avgPricePerItem add karo

JavaScript

```
db.orders.aggregate([
  {
    $addFields: {
      avgPricePerItem: { $divide: ["$totalAmount", "$quantity"] }
    }
  }
])
```

C) Students me feeStatus add karo

fees > 90000 → "High Fees"

warna "Normal Fees"

JavaScript

```
db.students.aggregate([
  {
    $addFields: {
      feeStatus: {
        $cond: [
          { $gt: ["$fees", 90000] },
          "High Fees",
          "Normal Fees"
        ]
      }
    }
  }
])
```

D) Posts me tagsCount add karo

JavaScript

```
db.posts.aggregate([
  {
    $addFields: {
      tagsCount: { $size: "$tags" }
    }
  }
])
```

E) Posts me commentsCount add karo

JavaScript

```
db.posts.aggregate([
  {
    $addFields: {
      commentsCount: { $size: "$comments" }
    }
  }
])
```

2) $count

Meaning

Pipeline ke end tak jo documents bache hain, unki **count** return karta hai.

A) Total students count

JavaScript

```
db.students.aggregate([
  { $count: "totalStudents" }
])
```

B) Indore students count

JavaScript

```
db.students.aggregate([
  { $match: { city: "Indore" } },
  { $count: "indoreStudents" }
])
```

C) Delivered orders count

JavaScript

```
db.orders.aggregate([
  { $match: { status: "Delivered" } },
  { $count: "deliveredOrders" }
])
```

29e-0780ef2b4e4f

3) `$facet`

Meaning

Ek hi pipeline me multiple parallel pipelines chala sakte ho.
Matlab ek hi query me multiple reports.

Ye interview me strong topic hai.

Example 1: Employees ka multi-report

- IT employees
- top salary employees
- department-wise avg salary

JavaScript

```javascript
db.employees.aggregate([
  {
    $facet: {
      itEmployees: [
        { $match: { department: "IT" } }
      ],

      top3Salary: [
        { $sort: { salary: -1 } },
        { $limit: 3 }
      ],

      avgSalaryByDept: [
        {
          $group: {
            _id: "$department",
            avgSalary: { $avg: "$salary" }
          }
        }
      ]
    }
  }
```

Example 2: Students ka multi-report

- Indore students
- top fees students
- city-wise count

JavaScript

```
db.students.aggregate([
  {
    $facet: {
      indoreStudents: [
        { $match: { city: "Indore" } }
      ],

      topFees: [
        { $sort: { fees: -1 } },
        { $limit: 5 }
      ],

      cityWiseCount: [
        {
          $group: {
            _id: "$city",
```

4) $bucket

Meaning

Documents ko **manual ranges / groups** me divide karta hai.
Jaise salary ranges, fees ranges, age ranges.

Syntax

JavaScript

```
{
  $bucket: {
    groupBy: "$field",
    boundaries: [ ... ],
    default: "Other",
    output: { ... }
  }
}
```

tGPT

Example 1: Students ko fees range me group karo

- 0 to 50000
- 50000 to 80000
- 80000 to 120000
- बाकी → Other

JavaScript

```
db.students.aggregate([
  {
    $bucket: {
      groupBy: "$fees",
      boundaries: [0, 50000, 80000, 120000],
      default: "Other",
      output: {
        totalStudents: { $sum: 1 },
        avgFees: { $avg: "$fees" }
      }
    }
  }
])
```

ıatGPT

Example 2: Employees salary buckets

JavaScript

```
db.employees.aggregate([
  {
    $bucket: {
      groupBy: "$salary",
      boundaries: [30000, 80000, 150000, 250000],
      default: "High Salary",
      output: {
        totalEmployees: { $sum: 1 },
        maxSalary: { $max: "$salary" }
      }
    }
  }
])
```

Example 3: Products rating buckets

JavaScript

```
db.products.aggregate([
  {
    $bucket: {
      groupBy: "$rating",
      boundaries: [1, 3, 5, 6],
      default: "Other",
      output: {
        totalProducts: { $sum: 1 }
      }
    }
  }
])
```

5) `$bucketAuto`

Meaning

MongoDB **automatically buckets** bana deta hai.
Hume boundaries manually nahi deni padti.

Example 1: Students fees ko 4 automatic buckets me divide karo

JavaScript

```
db.students.aggregate([
  {
    $bucketAuto: {
      groupBy: "$fees",
      buckets: 4
    }
  }
])
```

Example 2: Employees salary ko 5 auto buckets me divide karo

JavaScript

```
db.employees.aggregate([
  {
    $bucketAuto: {
      groupBy: "$salary",
      buckets: 5
    }
  }
])
```

Example 3: Orders totalAmount ko 3 buckets me divide karo

JavaScript

```
db.orders.aggregate([
  {
    $bucketAuto: {
      groupBy: "$totalAmount",
      buckets: 3
    }
  }
])
```

Difference: `$bucket` vs `$bucketAuto`

`$bucket`

- boundaries **hum manually** dete hain

`$bucketAuto`

- boundaries **MongoDB khud** decide karta hai

---

6) `$replaceRoot`

Meaning

Document ka **root replace** karta hai.
Mostly nested/embedded document ko root banane ke liye use hota hai.

Tumhare `companyDB` me best example:
`posts.comments` array of objects hai.

Example 1: comments ko unwind karke har comment ko root banao

JavaScript

```
db.posts.aggregate([
  { $unwind: "$comments" },
  {
    $replaceRoot: {
      newRoot: "$comments"
    }
  }
])
```

Example 2: comment root + count by user

JavaScript

```
db.posts.aggregate([
  { $unwind: "$comments" },
  {
    $replaceRoot: {
      newRoot: "$comments"
    }
  },
  {
    $group: {
      _id: "$user",
      totalComments: { $sum: 1 }
    }
  }
])
```

Example 3: Agar future me embedded object ho, usko root bana sakte ho

Example concept:

JavaScript

```
{
  name: "Tarun",
  address: {
    city: "Indore",
    pincode: 452010
  }
}
```

JavaScript

```
db.users.aggregate([
  {
    $replaceRoot: {
      newRoot: "$address"
    }
  }
])
```

7) `$sortByCount`

Meaning

Ye shortcut hai:

JavaScript

```
$group + $sort
```

Kisi field ke unique values ka count karke descending me sort karta hai.

Example 1: Students city-wise count

JavaScript

```
db.students.aggregate([
  { $sortByCount: "$city" }
])
```

Example 2: Employees department-wise count

JavaScript

```
db.employees.aggregate([
  { $sortByCount: "$department" }
])
```

Example 3: Orders status-wise count

JavaScript

```
db.orders.aggregate([
  { $sortByCount: "$status" }
])
```

Example 4: Tags count

`tags` array hai, so pehle unwind karna padega:

JavaScript

```
db.posts.aggregate([
  { $unwind: "$tags" },
  { $sortByCount: "$tags" }
])
```

Example 5: Comment users count

JavaScript

```
db.posts.aggregate([
  { $unwind: "$comments" },
  { $sortByCount: "$comments.user" }
])
```

JavaScript

```
use companyDB

// $addFields
db.employees.aggregate([
  {
    $addFields: {
      totalIncome: { $add: ["$salary", "$bonus"] }
    }
  }
])

db.posts.aggregate([
  {
    $addFields: {
      tagsCount: { $size: "$tags" },
      commentsCount: { $size: "$comments" }
    }
  }
])

// $count
```

```
// $count
db.students.aggregate([
  { $count: "totalStudents" }
])

db.orders.aggregate([
  { $match: { status: "Delivered" } },
  { $count: "deliveredOrders" }
])

// $facet
db.employees.aggregate([
  {
    $facet: {
      itEmployees: [{ $match: { department: "IT" } }],
      top3Salary: [{ $sort: { salary: -1 } }, { $limit: 3 }],
      avgSalaryByDept: [
        { $group: { _id: "$department", avgSalary: { $avg: "$salary" } } }
      ]
    }
  }
])
```

JavaScript

```javascript
// $bucket
db.students.aggregate([
  {
    $bucket: {
      groupBy: "$fees",
      boundaries: [0, 50000, 80000, 120000],
      default: "Other",
      output: {
        totalStudents: { $sum: 1 },
        avgFees: { $avg: "$fees" }
      }
    }
  }
])
```

```
// $bucketAuto
db.employees.aggregate([
  {
    $bucketAuto: {
      groupBy: "$salary",
      buckets: 5
    }
  }
])

// $replaceRoot
db.posts.aggregate([
  { $unwind: "$comments" },
  { $replaceRoot: { newRoot: "$comments" } }
])

// $sortByCount
db.students.aggregate([
  { $sortByCount: "$city" }
])
```

```
// $sortByCount
db.students.aggregate([
  { $sortByCount: "$city" }
])

db.posts.aggregate([
  { $unwind: "$tags" },
  { $sortByCount: "$tags" }
])
```

One-line interview definitions

- `$addFields` → document me new computed field add karta hai
- `$count` → pipeline result ka total count
- `$facet` → ek hi pipeline me multiple sub-pipelines
- `$bucket` → manual ranges me grouping
- `$bucketAuto` → automatic ranges me grouping
- `$replaceRoot` → root document replace karna
- `$sortByCount` → value-wise count + descending sort

Projection ka matlab:

query result me kaunse fields dikhane hain / kaunse hide karne hain.

Use:

JavaScript

```
use companyDB
```

---

1) Projection kya hota hai?

`find()` me 2nd argument projection hota hai.

Syntax

JavaScript

```
db.collection.find(
  { filter },
  { projection }
)
```

JavaScript

```
db.students.find(
  { city: "Indore" },
  { name: 1, city: 1 }
)
```

Isme result me mostly `name`, `city` aur by default `_id` aayega.

---

2) Include Projection

`1` ka matlab field **show/include** karna.

Example 1: students me sirf name aur city

JavaScript

```
db.students.find(
  {},
  { name: 1, city: 1 }
)
```

Example 2: employees me name, company, salary

JavaScript

```
db.employees.find(
  {},
  { name: 1, company: 1, salary: 1 }
)
```

Example 3: products me name, brand, price

JavaScript

```
db.products.find(
  {},
  { name: 1, brand: 1, price: 1 }
)
```

3) `_id` by default aata hai

Agar tum include projection karte ho, tab bhi `_id` default aata hai.

Example

JavaScript

```
db.students.find(
  {},
  { name: 1, city: 1 }
)
```

4) `_id` ko hide karna

Agar `_id` nahi chahiye to explicitly `0` likho.

Example

JavaScript

```
db.students.find(
  {},
  { _id: 0, name: 1, city: 1 }
)
```

5) Exclude Projection

`0` ka matlab field **hide/exclude** karna.

Example 1: students me fees aur dob hide

JavaScript

```
db.students.find(
  {},
  { fees: 0, dob: 0 }
)
```

Example 2: employees me bonus hide

JavaScript

```
db.employees.find(
  {},
  { bonus: 0 }
)
```

6) Important Rule

Include aur exclude ko mix nahi kar sakte

Except `_id`

❌ Invalid

JavaScript

```
db.students.find(
  {},
  { name: 1, city: 1, fees: 0 }
)
```

Ye galat hai because include + exclude mix ho gaya.

7) Single field projection examples

Students me sirf name

JavaScript

```
db.students.find({}, { name: 1, _id: 0 })
```

Employees me sirf salary

JavaScript

```
db.employees.find({}, { salary: 1, _id: 0 })
```

Orders me sirf customerName aur totalAmount

JavaScript

```
db.orders.find({}, { customerName: 1, tota nt: 1, _id: 0 })
```

8) Multiple field projection examples

Students me name, city, college

JavaScript

```
db.students.find(
  {},
  { _id: 0, name: 1, city: 1, college: 1 }
)
```

Employees me name, department, post, salary

JavaScript

```
db.employees.find(
  {},
  { _id: 0, name: 1, department: 1, post: 1, salary: 1 }
)
```

9) Embedded document projection

Agar field nested ho to **dot notation** use hoti hai.

Tumhare current `companyDB` dataset me direct address object nahi hai, but concept interview ke liye important hai.

Suppose document:

JavaScript

```
{
  name: "Tarun",
  address: {
    city: "Indore",
    pincode: 452010
  }
}
```

Sirf address.city dikhana

JavaScript

```
db.users.find(
  {},
  { _id: 0, "address.city": 1 }
)
```

address.pincode hide karna

JavaScript

```
db.users.find(
  {},
  { "address.pincode": 0 }
)
```

10) Array field projection

Tumhare `posts` collection me `tags` array hai.

Sirf title aur tags dikhana

JavaScript

```
db.posts.find(
  {},
  { _id: 0, title: 1, tags: 1 }
)
```

tags hide karna

JavaScript

```
db.posts.find(
  {},
  { tags: 0 }
)
```

12) Filter + Projection together

Indore students me sirf name, city

JavaScript

```
db.students.find(
  { city: "Indore" },
  { _id: 0, name: 1, city: 1 }
)
```

salary > 100000 employees me sirf name, salary

JavaScript

```
db.employees.find(
  { salary: { $gt: 100000 } },
  { _id: 0, name: 1, salary: 1 }
)
```

929e-078de12b4e41

Delivered orders me customerName aur totalAmount

JavaScript

```
db.orders.find(
  { status: "Delivered" },
  { _id: 0, customerName: 1, totalAmount: 1 }
)
```

---

13) Projection with cursor methods

Top 5 salary employees me sirf name, salary

JavaScript

```
db.employees.find(
  {},
  { _id: 0, name: 1, salary: 1 }
).sort({ salary: -1 }).limit(5)
```

14) Array element projection (advanced note)

Agar array ka specific part nikalna ho to aggregation ya operators lagte hain, but basic interview level par tumhe ye yaad rakhna hai:

- full array dikhana → `tags: 1`
- embedded array ke field dikhana → `"comments.user": 1`

```
// include projection
db.students.find({}, { name: 1, city: 1 })
db.employees.find({}, { name: 1, company: 1, salary: 1 })

// hide _id
db.students.find({}, { _id: 0, name: 1, city: 1 })

// exclude projection
db.students.find({}, { fees: 0, dob: 0 })
db.employees.find({}, { bonus: 0 })

// filter + projection
db.students.find(
  { city: "Indore" },
  { _id: 0, name: 1, city: 1 }
)

db.orders.find(
  { status: "Delivered" },
  { _id: 0, customerName: 1, totalAmount:
)
```

```
// array projection
db.posts.find({}, { _id: 0, title: 1, tags: 1 })

// embedded array projection
db.posts.find({}, { _id: 0, "comments.user": 1 })
db.posts.find({}, { _id: 0, "comments.msg": 1 })
```

B) Negative increment = decrement

product stock -2

JavaScript

```
db.products.updateOne(
  { name: "Product 10" },
  { $inc: { stock: -2 } }
)
```

C) Multiple numeric fields increment

JavaScript

```
db.payments.updateOne(
  { name: "Rahul Sharma" },
  {
    $inc: {
      salary: 2000,
      bonus: 500
    }
  }
)
```

4) `$mul`

Meaning

Numeric field ko given value se **multiply** karta hai.

Example 1: fees double

JavaScript

```
db.students.updateOne(
  { name: "Priya Verma" },
  { $mul: { fees: 2 } }
)
```

Example 2: salary 10% factor se multiply

JavaScript

```
db.employees.updateOne(
  { name: "Rahul Sharma" },
  { $mul: { salary: 1.1 } }
)
```

5) `$min`

Meaning

Field ko **given value aur existing value me jo minimum ho** usse set karta hai.

Example 1

Agar Rahul ki fees 70000 hai aur hum 65000 dete hain to 65000 ho jayegi.

JavaScript

```
db.students.updateOne(
  { name: "Rahul Sharma" },
  { $min: { fees: 65000 } }
)
```

Example 2

Agar existing salary 90000 hai aur hum 120000 dete hain to change nahi hoga.

JavaScript

```
db.employees.updateOne(
  { name: "Tarun Patidar" },
  { $min: { salary: 120000 } }
)
```

6) `$max`

Meaning

Field ko **given value aur existing value me jo maximum ho** usse set karta hai.

Example 1

JavaScript

```
db.students.updateOne(
  { name: "Priya Verma" },
  { $max: { fees: 95000 } }
)
```

7) `$rename`

Meaning

Field ka **name change** karta hai.

Example 1: students me `mobile` ko `phone` karo

JavaScript

```
db.students.updateMany(
  {},
  { $rename: { mobile: "phone" } }
)
```

8) `$currentDate`

Meaning

Field me **current date/time** set karta hai.

Example 1: student me lastUpdated add karo

JavaScript

```
db.students.updateOne(
  { name: "Rahul Sharma" },
  { $currentDate: { lastUpdated: true } }
)
```

Example 3: timestamp type

JavaScript

```
db.orders.updateOne(
  { _id: 2 },
  { $currentDate: { updatedAt: { $type: "timestamp" } } }
)
```

---

9) `$setOnInsert`

Meaning

Ye **sirf upsert ke time** kaam karta hai.

Agar document insert hoga tab field set karega.

Agar document already mil gaya aur sirf update hua, to `$setOnInsert` apply nahi hoga.

Example 1: new student insert if not found

JavaScript

```
db.students.updateOne(
  { name: "New Student" },
  {
    $set: { city: "Indore" },
    $setOnInsert: {
      age: 20,
      fees: 50000,
      college: "Medicaps Indore"
    }
  },
  { upsert: true }
)
```

Case:

- agar `"New Student"` nahi mila → new doc insert hoga
- usme `city`, `age`, `fees`, `college` sab set honge

Example 2: employee upsert

JavaScript

```
db.employees.updateOne(
  { name: "Aman Verma" },
  {
    $set: { department: "IT" },
    $setOnInsert: {
      salary: 40000,
      bonus: 5000,
      city: "Indore"
    }
  },
  { upsert: true }
)
```

Combined update examples

1) salary set + currentDate

JavaScript

```javascript
db.employees.updateOne(
  { name: "Tarun Patidar" },
  {
    $set: { salary: 150000 },
    $currentDate: { updatedAt: true }
  }
)
```

2) fees increment + max

JavaScript

```javascript
db.students.updateOne(
  { name: "Rahul Sharma" },
  {
    $inc: { fees: 5000 },
    $max: { fees: 90000 }
  }
)
```

3) set + unset together

JavaScript

```
db.students.updateOne(
  { name: "Priya Verma" },
  {
    $set: { city: "Delhi" },
    $unset: { pincode: "" }
  }
)
```

4) upsert + setOnInsert + currentDate

JavaScript

```
db.students.updateOne(
  { name: "Karan Joshi" },
  {
    $set: { city: "Bhopal" },
    $setOnInsert: {
      age: 21,
      fees: 60000
    },
    $currentDate: {
      updatedAt: true
    }
  },
  { upsert: true }
)
```

- `$set` → field set/update/create
- `$unset` → field remove
- `$inc` → numeric value increase/decrease
- `$mul` → numeric value multiply
- `$min` → smaller value set karega
- `$max` → bigger value set karega
- `$rename` → field name change
- `$currentDate` → current date/timestamp set
- `$setOnInsert` → sirf upsert insert ke time set hota hai

---

1) Index kya hota hai?

Upg

Index database ko **fast search** karne me help karta hai.

Without index:

- MongoDB ko **collection scan** karna padta hai
- matlab almost sab documents dekhne pad sakte hain

With index:

- MongoDB directly relevant documents tak pahunch sakta hai

Simple line for interview

Index is a data structure that improves query performance by reducing the number of documents MongoDB needs to scan.

2) Index kyu use karte hain?

Mostly in cases:

- `find()` fast karna
- `sort()` fast karna
- `filter + sort` fast karna
- unique values enforce karna
- text search karna

---

3) Basic syntax

Create index

JavaScript

```
db.collection.createIndex({ fieldName: 1 })
```

4) Single Field Index

Example: students collection me city par index

JavaScript

```
db.students.createIndex({ city: 1 })
```

Ab ye query fast ho sakti hai:

JavaScript

```
db.students.find({ city: "Indore" })
```

Example: employees salary par index

JavaScript

```
db.employees.createIndex({ salary: 1 })
```

Useful for:

JavaScript

```
db.employees.find({ salary: { $gt: 100000 } })
db.employees.find().sort({ salary: 1 })
```

5) Descending Index

JavaScript

```
db.orders.createIndex({ totalAmount: -1 })
```

Useful for:

JavaScript

```
db.orders.find().sort({ totalAmount: -1 })
```

6) `getIndexes()` — collection ke indexes dekhna

JavaScript

```
db.students.getIndexes()
```

Default output me `_id` index hota hi hai.

---

7) `_id` index

MongoDB har collection me `_id` par automatically index banata hai.

Example:

JavaScript

```
db.students.getIndexes()
```

Tumhe `_id_` naam ka default index dikhega.

8) Compound Index

Jab ek se zyada fields par index banate hain.

Example: employees me city + department

JavaScript

```
db.employees.createIndex({ city: 1, department: 1 })
```

Useful query:

JavaScript

```
db.employees.find({ city: "Indore", department: "IT" })
```

9) Compound Index ka important rule

Ye order matter karta hai.

Agar index hai:

JavaScript

```
db.employees.createIndex({ city: 1, department: 1 })
```

To ye queries useful ho sakti hain:

JavaScript

```
db.employees.find({ city: "Indore" })
db.employees.find({ city: "Indore", department: "IT" })
```

10) Unique Index

Agar kisi field me duplicate allow nahi chahiye.

Example: users email unique

JavaScript

```
db.users.createIndex({ email: 1 }, { unique: true })
```

Ab same email dobara insert nahi hoga.

Example: payments transactionId unique

JavaScript

```
db.payments.createIndex({ transactionId: 1 }, { unique: true })
```

11) Named Index

Khud ka custom naam de sakte ho.

JavaScript

```
db.students.createIndex(
  { city: 1 },
  { name: "city_index" }
)
```

Check:

JavaScript

```
db.students.getIndexes()
```

9e-0780ef2b4e4f

12) Drop Index

Single index delete

JavaScript

```
db.students.dropIndex("city_index")
```

Ya agar auto-generated name ho:

JavaScript

```
db.students.dropIndex("city_1")
```

---

13) Drop all indexes

JavaScript

```
db.students.dropIndexes()
```

14) Explain — interview me bahut important

Ye batata hai query ka execution plan kya hai.

Example

JavaScript

```
db.students.find({ city: "Indore" }).explain("executionStats")
```

Isse pata chalega:

- query ne **COLLSCAN** kiya ya **IXSCAN**
- kitne docs scan hue
- kitne docs return hue

16) Example without index

JavaScript

```
db.students.find({ college: "Medicaps Indore" }).explain("executionStats")
```

Agar `college` par index nahi hai, to often `COLLSCAN` aayega.

---

17) Example with index

JavaScript

```
db.students.createIndex({ college: 1 })

db.students.find({ college: "Medicaps Indore" }).explain("executionStats")
```

18) Sort + Index

Agar tum sort kar rahe ho, to sort field par index helpful hota hai.

Example

JavaScript

```
db.products.createIndex({ price: 1 })

db.products.find().sort({ price: 1 })
```

Example 2

JavaScript

```
db.employees.createIndex({ salary: -1 })

db.employees.find().sort({ salary: -1 }).l 5)
```

19) Filter + Sort ke liye compound index

Ye bahut important case hai.

Query

JavaScript

```
db.orders.find({ status: "Delivered" }).sort({ totalAmount: -1 })
```

Iske liye best index:

JavaScript

```
db.orders.createIndex({ status: 1, totalAmount: -1 })
```

20) Text Index

Agar text search karni ho.

Example: posts message par text index

JavaScript

```
db.posts.createIndex({ message: "text" })
```

Search:

JavaScript

```
db.posts.find({ $text: { $search: "MongoDB" } })
```

21) TTL Index

TTL index automatically old documents expire/delete kar sakta hai.
Interview me puchh sakte hain, so basic idea yaad rakhna.

Example concept

JavaScript

```
db.sessions.createIndex(
  { createdAt: 1 },
  { expireAfterSeconds: 3600 }
)
```

Matlab 1 hour ke baad docs expire ho sakte hain.

> Tumhare current dataset me practical use kam hai, but concept important hai.

22) Sparse Index

Sirf un documents par index banata hai jisme field present ho.

JavaScript

```
db.users.createIndex(
  { middleName: 1 },
  { sparse: true }
)
```

23) Partial Index

Sirf kuch matching documents ke liye index banta hai.

Example: only active users

JavaScript

```
db.users.createIndex(
  { city: 1 },
  { partialFilterExpression: { isPass: true } }
)
```

24) Multikey Index

Agar field array hai, MongoDB automatically multikey index bana sakta hai.

Tumhare `posts.tags` array field par:

JavaScript

```
db.posts.createIndex({ tags: 1 })
```

Query:

JavaScript

```
db.posts.find({ tags: "mongodb" })
```

26) Index ka nuksan bhi hota hai

Indexes helpful hain, but:

- extra storage lete hain
- insert/update/delete thoda slow ho sakta hai
- bahut zyada unnecessary indexes nahi banane chahiye

Interview line:

Indexes speed up reads but add overhead to writes and storage.

27) Tumhare `companyDB` ke liye kaunse indexes practical hain?

Upgr

`students`

JavaScript

```
db.students.createIndex({ city: 1 })
db.students.createIndex({ college: 1 })
db.students.createIndex({ city: 1, college: 1 })
```

`employees`

JavaScript

```
db.employees.createIndex({ company: 1 })
db.employees.createIndex({ department: 1 })
db.employees.createIndex({ city: 1, department: 1 })
db.employees.createIndex({ salary: -1 })
```

C) unique index

JavaScript

```
db.users.createIndex({ email: 1 }, { unique: true })
```

D) filter + sort index

JavaScript

```
db.orders.createIndex({ status: 1, totalAmount: -1 })

db.orders.find({ status: "Delivered" })
         .sort({ totalAmount: -1 })
         .explain("executionStats")
```

- **Single field index** → ek field par index
- **Compound index** → multiple fields par index
- **Unique index** → duplicate values allow nahi
- **Text index** → text search ke liye
- **Multikey index** → array field ke liye
- **TTL index** → time ke baad auto expire
- **Explain** → query execution plan dikhata hai
- **IXSCAN** → index use hua
- **COLLSCAN** → pura collection scan hua